

Exame de Programação Funcional

11 de Junho de 2012

DCC/FCUP

Versão A

Nome: _____

Nº mecanográfico: _____

- Este exame contém 6 questões em 4 páginas.
- Responda às questões 1, 2, 3 no espaço marcado no enunciado.
- Responda às questões 4, 5 e 6 numa folha de exame separada.
- Duração: 2h + 30m tolerância.

1. (15%) Responda a cada uma das seguintes questões, indicando **apenas** o resultado de cada expressão.

(a) `head (reverse [1,2,3])` = _____

(b) `map (*2) [0,1,2]` = _____

(c) `filter (>0) [2,-1,1,0,-2]` = _____

(d) `sum [x*y | (x,y)<-zip [1,2] [-2,2]]` = _____

(e) `foldr (\x y->2*x+y) 0 [1,2,3]` = _____

(f) Indique um tipo admissível para `[[1],[2,3],[4,5,6]]`:

(g) Indique um tipo admissível para `\xs->filter (/=' ') xs`:

(h) Considere a seguinte definição da função `elem` do prelúdio-padrão.

```
elem x [] = False
elem x (y:ys) = x==y || elem x ys
```

Indique o tipo mais geral desta definição:

2. (20%) Escreva uma definição recursiva da função `duplicar :: String -> String` que repete duas vezes cada vogal (letras 'a', 'e', 'i', 'o', 'u' minúsculas ou maiúsculas) numa cadeia de caracteres; os outros caracteres devem ficar inalterados.

Exemplo: `duplicar "--01a, mundo!" = "--001aa, muundoo!"`.

3. (20%) Considere uma representação de pontos no plano pelo par das suas coordenadas cartesianas:

```
type Ponto = (Float, Float)
```

- (a) Escreva uma definição da função que calcula a distância euclidiana entre dois pontos (x_1, y_1) e (x_2, y_2) : $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

```
dist :: Ponto -> Ponto -> Float
```

- (b) Considere agora um percurso dado como uma lista de pontos consecutivos. Escreva uma função `comprimento :: [Ponto] -> Float` que calcule o *comprimento total dum percurso* (isto é, a soma das distâncias entre pontos consecutivos).

Pode usar recursão ou listas em compreensão e deve usar a função `dist` da alínea anterior para calcular as distâncias. Tenha atenção de tratar corretamente os percursos degenerados (vazios ou com apenas um ponto).

**Responda às questões 4, 5 e 6 numa folha de exame separada.
Indique na folha qual a versão do exame (A ou B).**

4. (15%) Considere a definição em Haskell dum tipo de dados para árvores binárias de pesquisa:

$$\text{data Arv } a = \text{Vazia} \mid \text{No } a \text{ (Arv } a) \text{ (Arv } a)$$

- (a) Escreva a definição da função `insert :: Ord a => a -> Arv a -> Arv a` para inserir um valor numa árvore mantendo a ordenação e sem repetições.
- (b) Desenhe o diagrama da árvore resultante da expressão

`inserir 3 (inserir 1 (inserir 2 Vazia))`

Diga se a árvore é equilibrada, justificando a sua resposta.

5. (15%) Considere a definição da função *replicate* do prelúdio-padrão de Haskell:

$$\text{replicate } 0 \ x = [] \quad (\text{repl.1})$$
$$\text{replicate } n \ x \mid n > 0 = x : \text{replicate } (n - 1) \ x \quad (\text{repl.2})$$

Usando indução sobre n , mostre que $\text{length } (\text{replicate } n \ x) = n$ para qualquer x e $n \geq 0$.

Sugestão: comece por escrever uma definição recursiva de *length*.

6. (15%) Usado indução sobre listas, mostre que

$$\text{map } f \ (\text{reverse } xs) = \text{reverse } (\text{map } f \ xs)$$

para qualquer função f e lista finita xs .

Sugestão: comece por escrever definições recursivas das funções *map* e *reverse*; pode ainda usar o seguinte resultado auxiliar (dum exercício do segundo teste):

$$\text{map } f \ (xs \ ++ \ ys) = \text{map } f \ xs \ ++ \ \text{map } f \ ys$$